

Lightweight RPC for Kotlin Multiplatform

Good day. My name is Aleksandr Stupnikov. The topic of my master thesis is "Lightweight RPC for Kotlin Multiplatform".

Problem Statement and Motivation

Kotlin is widely used for network-heavy software — backend microservices, client-server applications, distributed systems. Many of these projects follow a shared-codebase model, where client and server live in the same Kotlin project and share data types and business logic. The code in such a project naturally splits into two parts: business logic, which is unique to each application, and network code, which is repetitive across almost every application. Routing, serialization, HTTP client setup, error handling — this is boilerplate that every developer writes again and again. The problem is that even in a shared codebase, where both sides are in the same project, this network part remains unnecessarily large.

Ktor Boilerplate

The most common way to handle network communication in Kotlin is Ktor. With Ktor, the developer manually writes routing logic, serialization and deserialization of request and response bodies, HTTP client calls, and error handling. This gives fine-grained control, which is valuable when building a public API. But for internal service-to-service communication in a shared codebase, none of this control is needed, and all of it becomes pure overhead.

kRPC and gRPC Boilerplate

gRPC and Kotlin RPC follow the Remote Method Invocation pattern: the developer declares a service as an interface, implements it in a class, and receives a client proxy. In a shared codebase this pattern has three specific drawbacks. First, the developer must declare and implement a service interface just to make functions remotely callable — in a shared-codebase project the function signature is already the contract, so the interface duplicates information that is already there. Second, top-level functions cannot be remote at all. Third, at the call site a remote call looks identical to a local call — nothing tells the developer this is a network operation that may fail or be slow. These are the three problems this thesis addresses.

Background — Context Parameters

Before describing the solution, let me briefly explain the Kotlin feature it is built on. Context parameters are an experimental Kotlin feature that lets a function declare implicit parameters resolved from the enclosing scope. In this example, `greet` requires a `Logger`, but the caller does not pass it explicitly — it is resolved automatically from the enclosing context block. The parameter is part of the function signature, so the requirement is visible in the type. It is resolved automatically at the call site. And if no matching context exists in scope, the compiler reports an error. This gives a typed, compile-checked way to express that a function depends on something from its environment.

Goals and Objectives

Armed with this background, I can state the goal. The goal is to develop an RPC framework for Kotlin Multiplatform shared-codebase projects that reduces boilerplate compared to existing approaches. The three objectives are: first, design an RPC framework that uses context parameters as the mechanism for expressing remote calls; second, implement the framework as a Kotlin compiler plugin together with a runtime library; third, evaluate the result by comparing it with Kotlin RPC on example projects.

Context Parameters for RPC

The key idea is to use a context parameter to carry remote execution configuration. A function annotated with `@Remote` declares a context parameter of type `RemoteContext`. This context determines where the function executes — locally or on a remote server — and carries the information needed to make the network call. Because the parameter appears in the function signature, the remoteness of the function is visible at the type level. The developer cannot write a remote call without an enclosing context block to provide that parameter, so the remoteness is always explicit.

RemoteContext Implementation

RemoteContext is a sealed interface with two implementations. LocalContext means "run the original body in the current process". ConfiguredContext wraps a RemoteConfig, which holds the HTTP client configured with the server address. The covariance of the type parameter T and the fact that Nothing is Kotlin's bottom type make LocalContext a subtype of RemoteContext for any T. The consequence is that any remote function can always be called in LocalContext and will type-check. On the server, the framework provides LocalContext for all function invocations, so the original body runs directly with no network call.

How It Looks to the User

Here is the full developer experience. The user writes a normal Kotlin function, adds the `@Remote` annotation and a context parameter — that is the entire declaration overhead. On the client side, the user opens a context block with a `RemoteConfig` object that holds the Ktor `HttpClient` pointing at the server. Inside that block, `multiply` is called exactly like a local function. The framework handles the serialization, network call, and deserialization. On the server, the same function declaration is used — the framework calls it inside a `LocalContext`, and the original body executes directly. One function, two execution modes, no duplication.

What Compiler Plugin Generates

The transformation is done entirely by a Kotlin compiler plugin at the IR level. When the plugin encounters `@Remote`, it wraps the original body in a conditional: if the context is `LocalContext`, the original body runs — this is the server path. Otherwise the context must be a `ConfiguredContext`, and the plugin generates code that packages the function name and arguments into a `RemoteCall` and sends it using the HTTP client from the config. This is completely automatic — the user never writes this branching code. The plugin also replaces `genCallableMap()` calls with statically generated metadata for every `@Remote` function in the compilation unit, which is how the framework works on non-JVM KMP targets where reflection is unavailable.

Ktor Integration

The framework core is transport-agnostic. The `RemoteClient` interface is the only transport boundary, and an alternative transport can be plugged in by providing a custom implementation. For the default case we ship a Ktor integration: the `KRemote` plugin handles incoming remote calls on the server side, and the `remoteClient` extension wraps an `HttpClient` for the client side. Because the transport is Ktor, the full Ktor ecosystem — authentication, logging, CORS, rate limiting — applies to remote calls automatically.

Example Applications

To evaluate the framework I developed three example applications, each written twice — once with Kotlin Remote and once with Kotlin RPC. Todo is a simple CRUD app with four remote operations that establishes the per-application baseline cost. Social platform splits a backend into ten microservices with thirty-six remote operations and stresses per-microservice overhead and cross-service orchestration. CMS has fourteen operations split across four hierarchical capability tiers with Ktor basic authentication protecting each tier.

Framework-Specific Code per Application

Here are the boilerplate comparison results. Framework-specific lines are those that would not appear in an equivalent in-process program: on the Kotlin Remote side that means @Remote annotations, context parameter declarations, RemoteConfig definitions, and context switch blocks; on the Kotlin RPC side it means @Rpc interface declarations, implementation class shells, and withService and registerService call sites. On the smallest application, Todo with four operations, both frameworks produce exactly twelve lines — the API is too small for the structural difference to compound. On the Social platform with ten microservices the gap is 35%, because each microservice in Kotlin RPC requires an interface and an implementation class that have no counterpart in Kotlin Remote. On the CMS with four tiers the gap is 20%, driven by the four interface and class shells Kotlin RPC needs per tier.

Boilerplate example

Here is a concrete example from the Social platform. On the left, the Kotlin Remote version: two top-level functions with an `@Remote` annotation and a context parameter each. The business logic is in the function body. To call `getUser` from inside `postWithAuthor` on a different microservice, you wrap the call in a context block — one extra line. On the right, the Kotlin RPC version: an `@Rpc` interface and an implementation class for each service, and `PostsServiceImpl` must take a `UserService` stub as a constructor parameter. That stub has to be created before `PostsServiceImpl` is instantiated and threaded through its constructor. At ten microservices and thirty-six operations this pattern compounds into the 35% gap shown in the table.

Future work

Three directions were identified during development. Code slicing would use the RemoteConfig types — which already encode which node a function belongs to — to compile per-node artifacts containing only reachable code, so each service ships only the code it actually executes. Bidirectional streaming would add Flow-based communication and cover the current absence of a push-based API. Remote lambdas would allow function values to be remote callables, enabling higher-order remote functions where the operation is performed on the remote machine.

Summary

To summarize. We developed Kotlin Remote, an RPC framework for Kotlin Multiplatform shared-codebase projects. It uses context parameters to express remote calls without a service interface — the context parameter appears in the function signature making remoteness explicit, and is resolved implicitly at the call site. On larger applications the framework reduces framework-specific code by 35% for the Social platform and 20% for the CMS compared to Kotlin RPC. All three objectives were completed, and benchmarks confirm there is no framework-level performance overhead. Thank you for your attention. I am ready for questions.

Limitations

The framework has two main limitations. The first is scope: Kotlin Remote is designed for Kotlin-only shared-codebase projects. It does not support cross-language communication and is not meant for projects where client and server are developed independently — in those situations gRPC or Kotlin RPC remain the right tool. The second is the dependency on experimental Kotlin features: context parameters require the `-Xcontext-parameters` flag, and the compiler plugin API changes with each Kotlin release, requiring the plugin to be updated accordingly. There is also no streaming support — no Flow-based API.

Performance Comparison

Performance benchmarks were run with server and client in the same JVM process over loopback. The latency gap of roughly 50 to 100 microseconds comes from the transport choice: Kotlin Remote sends one HTTP/1.1 request per call, while Kotlin RPC multiplexes calls over a persistent WebSocket. This is a transport-level cost — a connection-reusing RemoteClient implementation would close most of it. Local dispatch when a @Remote function runs in LocalContext adds only 6.7 nanoseconds above a plain suspend call. Per-function bytecode is within 12% of Kotlin RPC. The boilerplate reduction does not come at a runtime or compiled-size penalty.

Feature Comparison

This table compares Kotlin Remote with gRPC and Kotlin RPC on the features that matter for choosing between them. Kotlin Remote does not require a service interface, supports top-level functions, makes remoteness visible through the enclosing context block, supports stateful remote objects, and allows hierarchical capability tiers through context subtyping. The main gaps compared to alternatives are no streaming support and no cross-language communication — both are deliberate scope decisions for the shared-codebase Kotlin setting.

RemoteSerializable classes — Distributed Objects

The framework also supports distributed objects. A class marked `@RemoteSerializable` can have `@Remote` methods. When such a class is returned from a remote function, the real instance is stored in a pool on the server and the client receives a generated stub — a subclass holding only an ID and the server URL. Subsequent method calls on the stub are forwarded to the server, which looks up the real instance by ID and calls the method on it, preserving state across calls. Constructors cannot have context parameters, so remote classes use a factory function or companion invoke operator annotated with `@Remote`.

Remote Classes — How It Works

When a `@RemoteSerializable` instance is serialized, the real object is added to `RemoteInstancesPool` on the server keyed by a generated ID, and the client receives the stub. When a method is called on the stub, it triggers a remote call passing the stub as the implicit `this` argument. The server deserializes the ID, looks up the real instance, and calls the method on it. This preserves mutable state across calls. To prevent memory leaks, the framework uses a lease-based garbage collector: the client periodically renews leases for stubs it holds, using weak references to detect when a stub has been collected locally. When the lease expires on the server, the instance is removed from the pool.